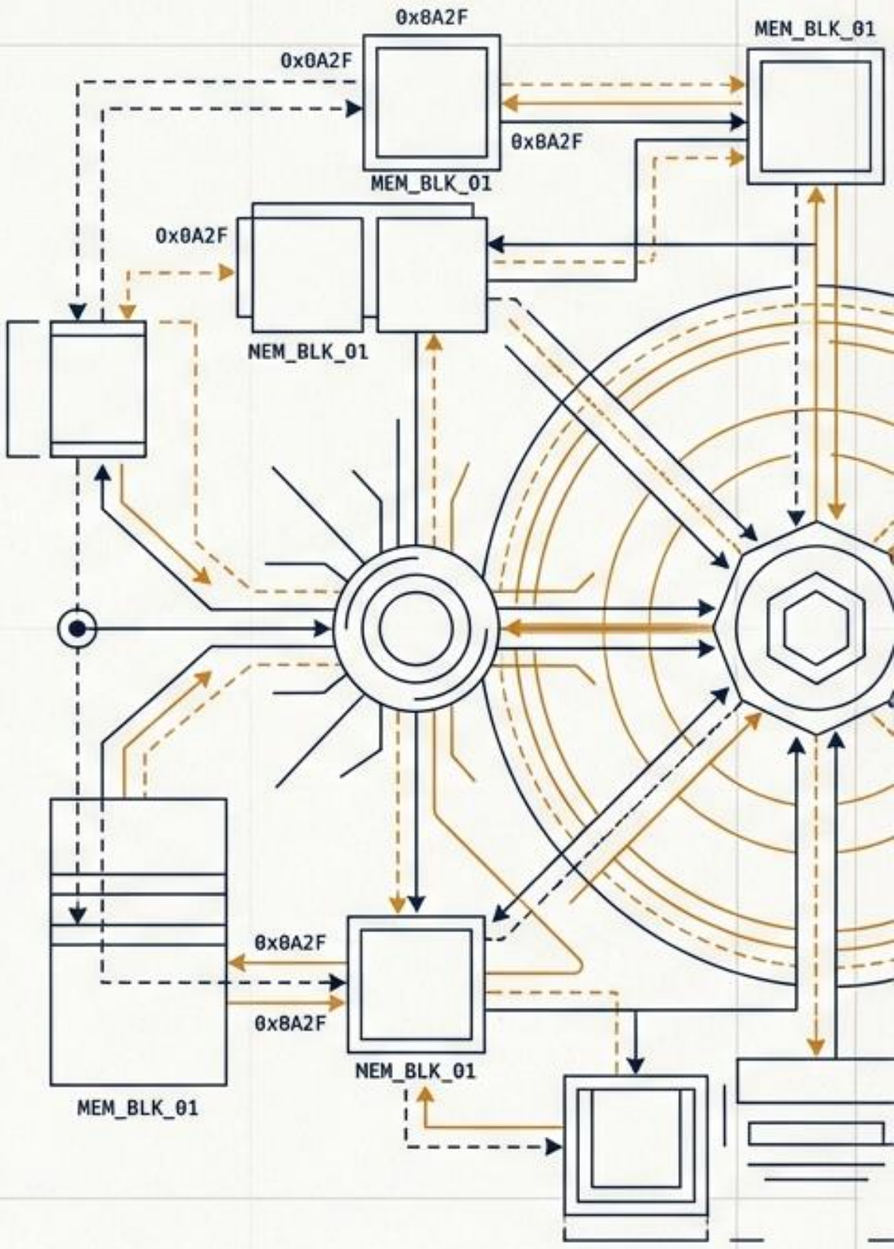
Chuyên đề: Quản lý & Giao tiếp Tiến Trình (Process Management & IPC)

Tài liệu Hướng dẫn Thảo luận Nhóm

Giảng viên: ThS Nguyễn Kiên Cường



Khung Thảo luận & Mục tiêu

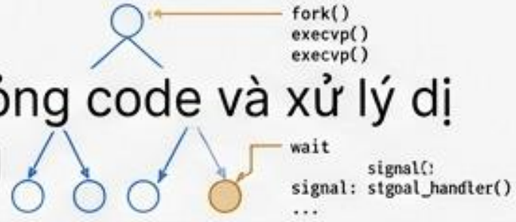
Module 1: Bản chất của Tiến trình

Nắm bắt kiến trúc bộ nhớ và vòng đời

Module 2: Điều phối & Ngữ cảnh

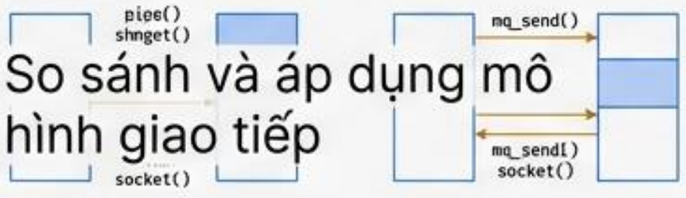
Phân tích chiến lược định thời CPU

Module 3: Thao tác & Gia phả



Mô phỏng code và xử lý dị thường

Module 4: Truyền thông IPC



So sánh và áp dụng mô hình giao tiếp

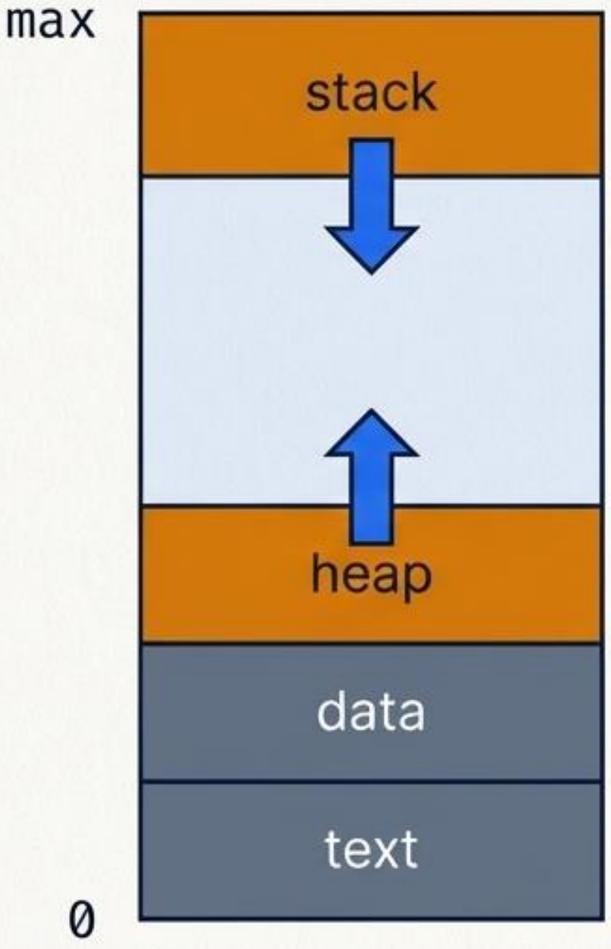
Khung Yêu cầu

Nguyên tắc Seminar:

1. Phân tích sơ đồ thay vì đọc chữ.
2. Giải quyết tình huống thực tế bằng lập luận kỹ thuật.
3. Đại diện nhóm bảo vệ quan điểm trước lớp.

Giải phẫu một Tiến trình (Process Anatomy)

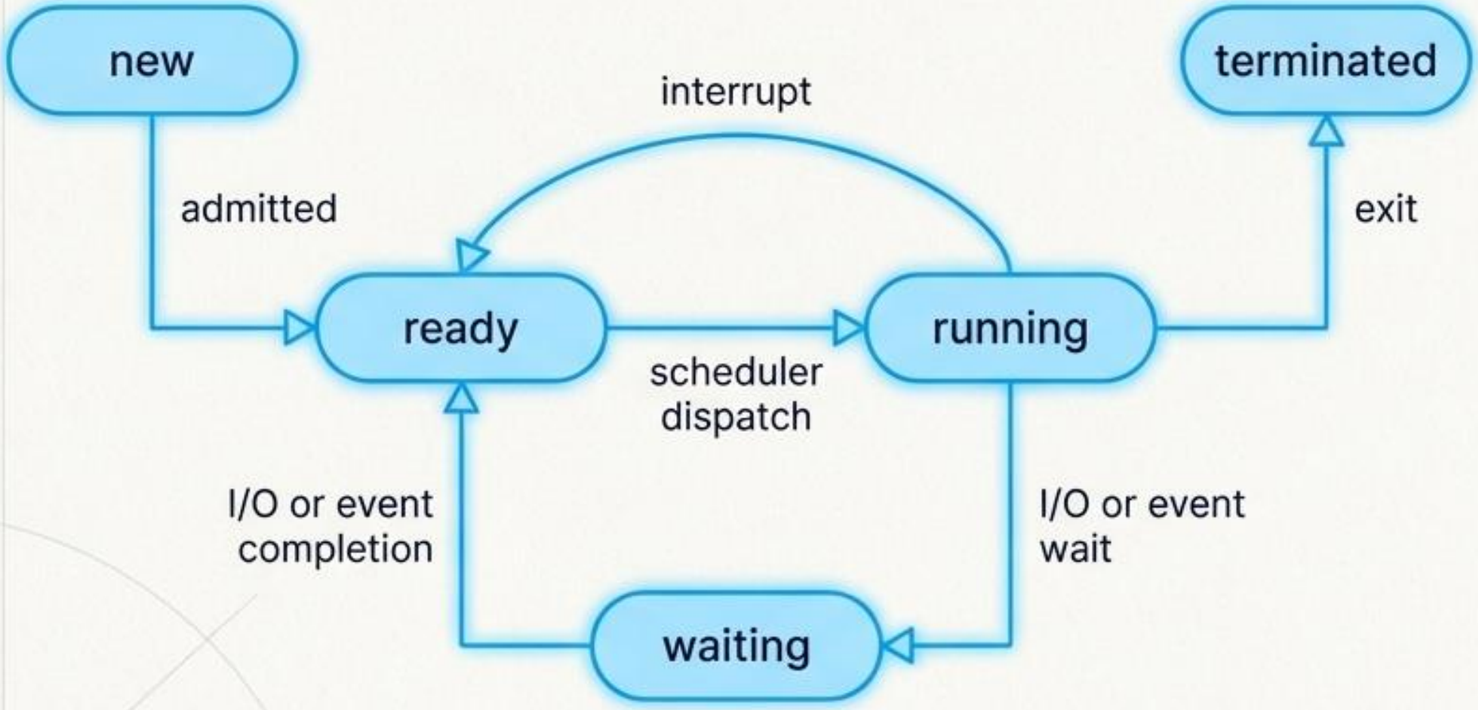
Chương trình là thực thể thụ động (file .exe trên đĩa).
Tiến trình là thực thể hoạt động (nạp vào bộ nhớ).



Nhiệm vụ Thảo luận

Dựa vào sơ đồ bộ nhớ bên trái, nhóm hãy giải thích tại sao hai người dùng cùng chạy chung một file .exe lại tạo ra hai tiến trình với vùng Stack/Heap hoàn toàn khác nhau? Điều gì xảy ra trong vùng Data?

Vòng đời & Trạng thái Tiến trình (Lifecycle & States)

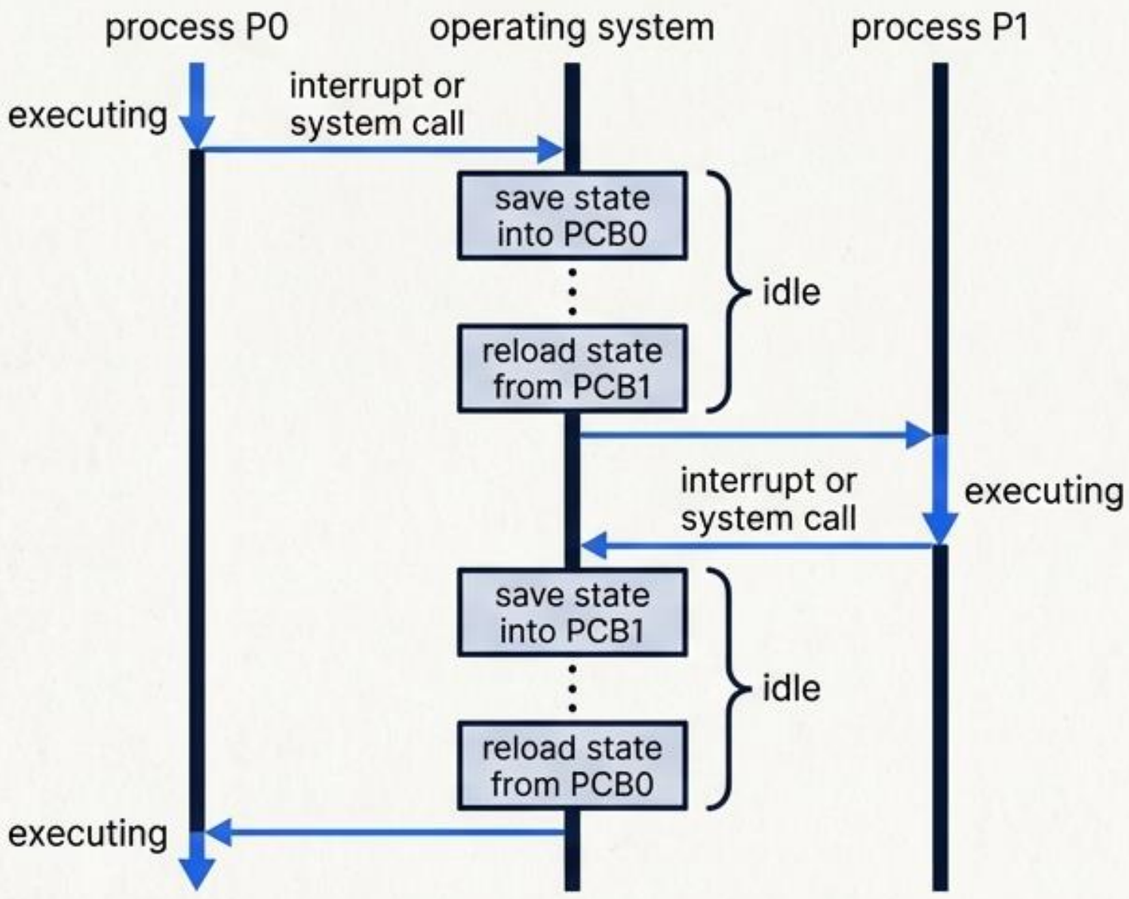


Nhiệm vụ Thảo luận

Xác định 3 sự kiện thực tế trong quá trình sử dụng máy tính (ví dụ: người dùng gõ phím, tải file hoàn tất, mở một tab mới) tương ứng với tương ứng với các mũi tên chuy' từ 'Running' sang 'Waiting' và từ 'Waiting' sang 'Ready'.

PCB & Chuyển ngữ cảnh (Context Switching)

Hệ thống không làm công việc hữu ích trong thời gian lưu/tải PCB.



process state
process number
program counter
registers
memory limits
list of open files
...

Nhiệm vụ Thảo luận

Thời gian CPU nhàn rỗi (idle) trong quá trình chuyển ngữ cảnh là một "chi phí" quản lý cốt lõi. Nhóm hãy đề xuất 2 giải pháp (có thể can thiệp từ phần cứng hoặc tối ưu phần mềm) để giảm thiểu thời gian chuyển ngữ cảnh này.

Ma trận các Bộ định thời (Schedulers)

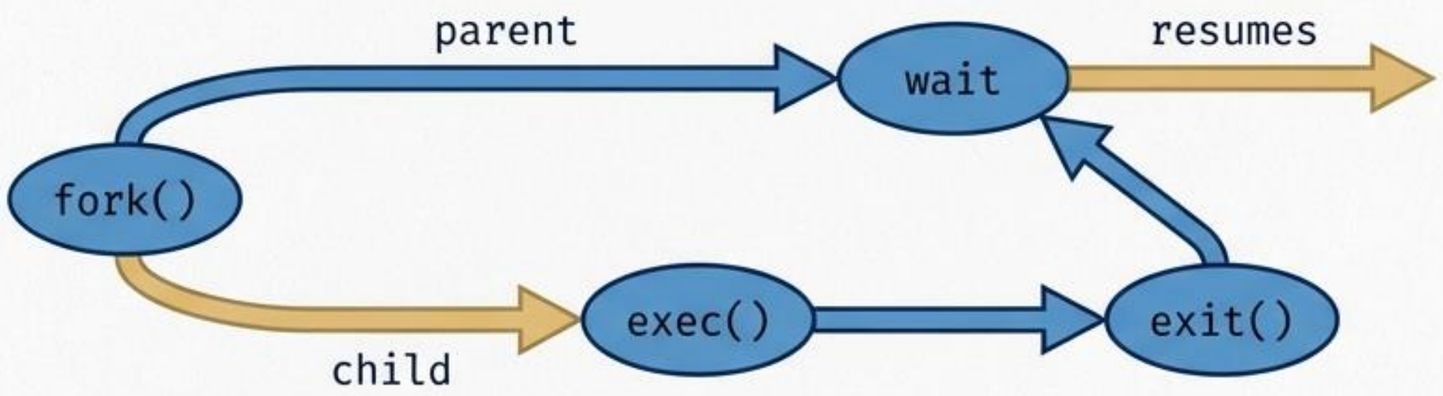
Ngắn hạn (Short-term)	Trung hạn (Medium-term)	Dài hạn (Long-term)
Chức năng: Chọn tiến trình cho CPU. **Tần suất:** Mili giây (rất nhanh).	**Chức năng:** Di chuyển tiến trình ra/vào đĩa (Swapping). **Tần suất:** Trung bình.	**Chức năng:** Đưa vào Ready queue. Kiểm soát mức độ đa chương. **Tần suất:** Giây/phút (chậm).

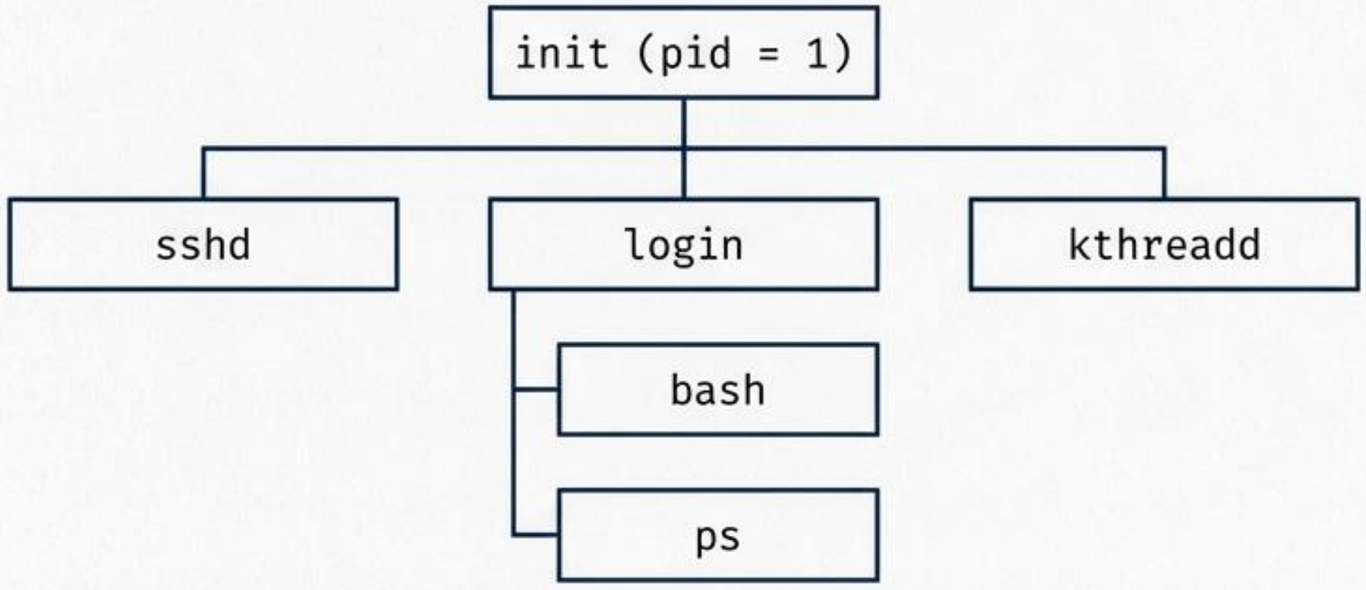
Nhiệm vụ Thảo luận

Phân tích sự khác biệt giữa tiến trình thiên hướng I/O (I/O-bound) và thiên hướng xử lý (CPU-bound). Bộ định thời dài hạn (Long-term Scheduler) cần kết hợp tỷ lệ hai loại tiến trình này như thế nào để tối ưu hóa toàn bộ hệ thống?

Cây Tiến trình & Lời gọi Hệ thống

Quản lý thông qua mã định danh (PID).





Nhiệm vụ Thảo luận

Dựa vào vòng lặp hệ thống, sự khác biệt cốt lõi về Không gian địa chỉ (Address Space) giữa việc tiến trình con chỉ sử dụng `fork()` so với việc sử dụng thêm lệnh `exec()` ngay sau đó là gì?

Lab Mô phỏng: Phân tích Lời gọi Hệ thống (System Call Simulation)

UNIX: fork()

```c
pid = fork();
if (pid < 0) { /* error occurred */
    fprintf(stderr, "Fork Failed");
}
} else if (pid == 0) { /* child process */
    execlp("/bin/ls", "ls", NULL);
}
} else { /* parent process */
    wait(NULL);
    printf("Child Complete");
}
```

Windows: CreateProcess()

```c
CreateProcess(NULL, "C:\\WINDOWS\\system32\\mspaint.exe",
     NULL, NULL, FALSE, 0, NULL, NULL, &si, &pi);

/* parent will wait for the child to complete */
WaitForSingleObject(pi.hProcess, INFINITE);
printf("Child Complete");
```

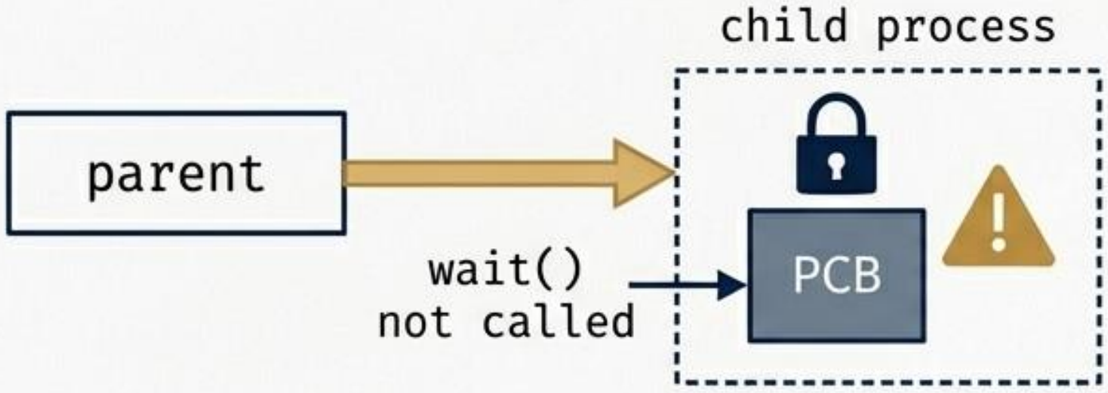
Nhiệm vụ Thảo luận

Giả lập thực thi đoạn code UNIX: Trình bày từng bước giá trị trả về của biến **pid**. Điều gì sẽ xảy ra trong terminal nếu **lập trình viên quên gọi hàm wait(NULL) ở tiến trình cha?**

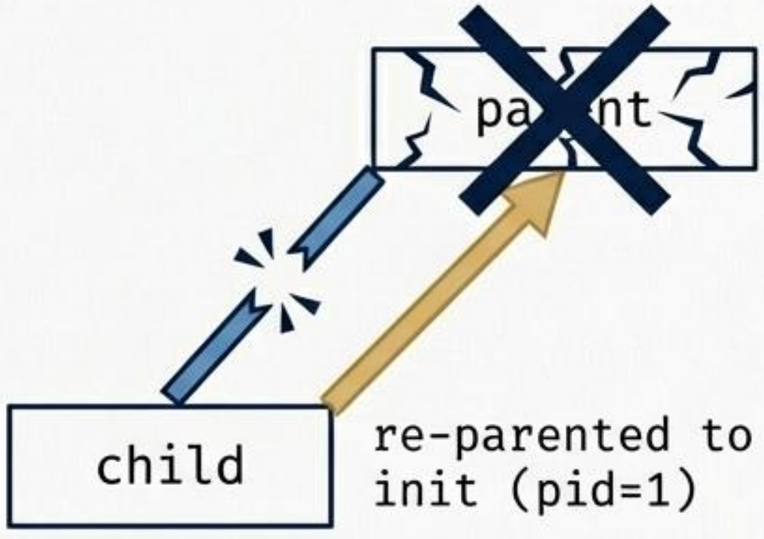
Các trạng thái Dị thường: Zombie & Orphan

Hệ thống có cơ chế "Kết thúc theo tầng" (Cascading termination).

Zombie Process



Orphan Process

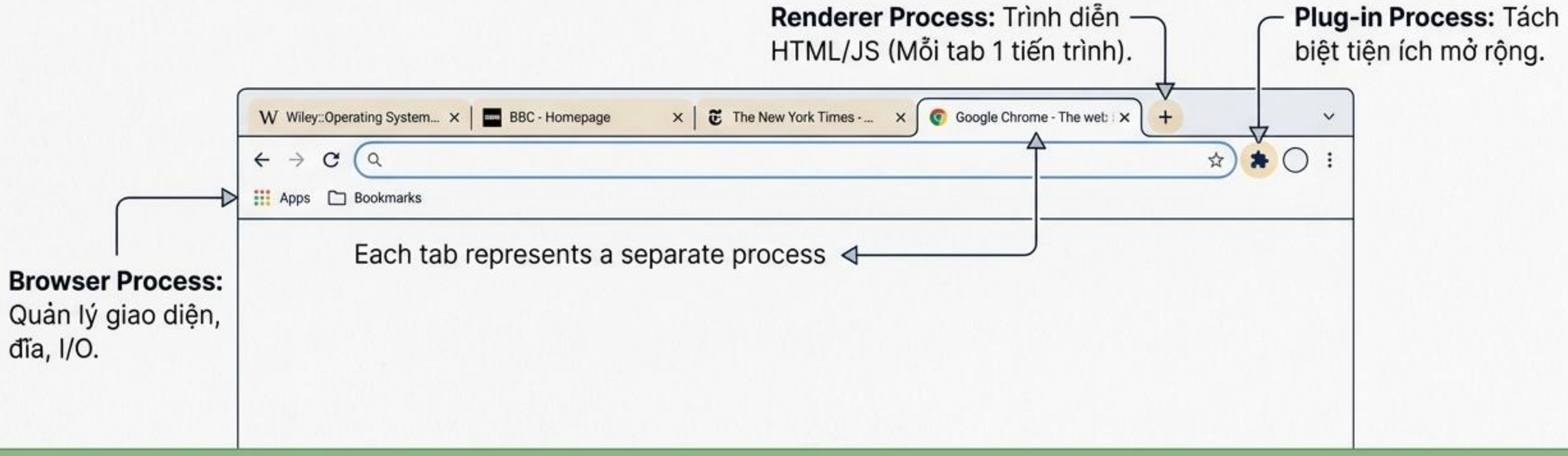


Nhiệm vụ Thảo luận

Trình bày rủi ro tài nguyên đối với hệ thống nếu có quá nhiều tiến trình Zombie xuất hiện cùng lúc.

Trình bày cách Hệ điều hành chủ động xử lý và 'nhận nuôi' một tiến trình Orphan như thế nào?

Case Study Thực tế: Kiến trúc Google Chrome



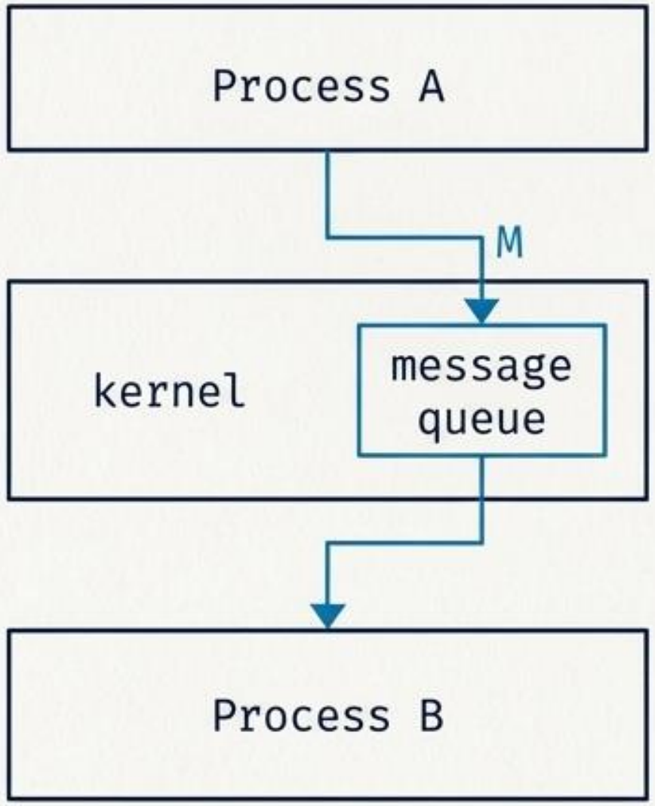
Nhiệm vụ Thảo luận

Tại sao Chrome lại chọn kiến trúc Đa tiến trình (Multi-process) thay vì Đơn tiến trình (Single-process) dù thiết kế này tiêu tốn nhiều bộ nhớ hơn rất nhiều? Phân tích quyết định này dưới góc độ 'Sandboxing' (Hộp cát bảo mật) và Quản lý lỗi.

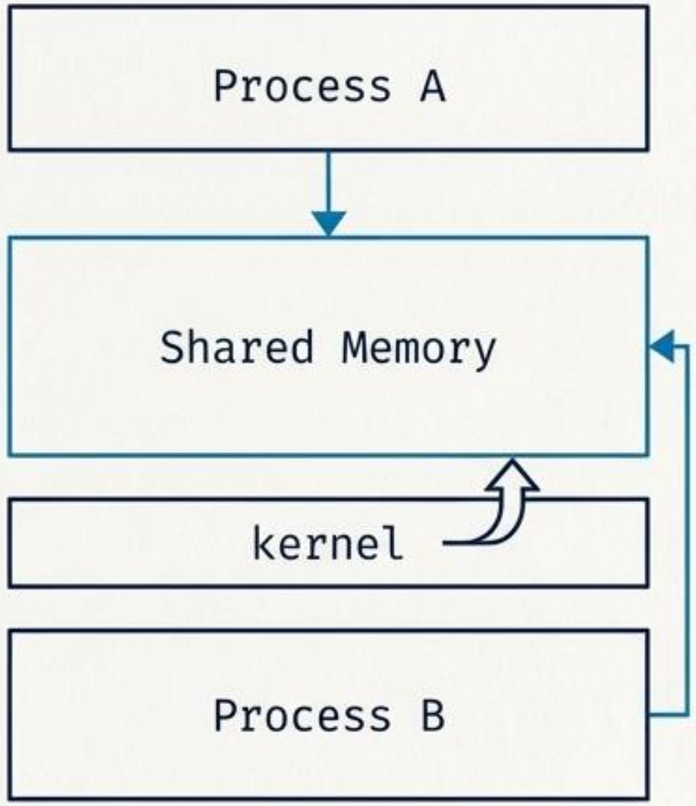
Nhu cầu Hợp tác & 2 Mô hình IPC

Tiến trình cộng tác: Chia sẻ thông tin, Tăng tốc tính toán, Đơn thể hóa.

Message Passing



Shared Memory



Nhiệm vụ Thảo luận

Giả sử hai tiến trình cần chia sẻ một khối lượng dữ liệu khổng lồ (ví dụ: render một file video 4K).

Trong hai mô hình (Message Passing vs. Shared Memory), mô hình nào sẽ mang lại hiệu suất băng thông cao hơn? Tại sao?

Bảng so sánh chuẩn hóa IPC

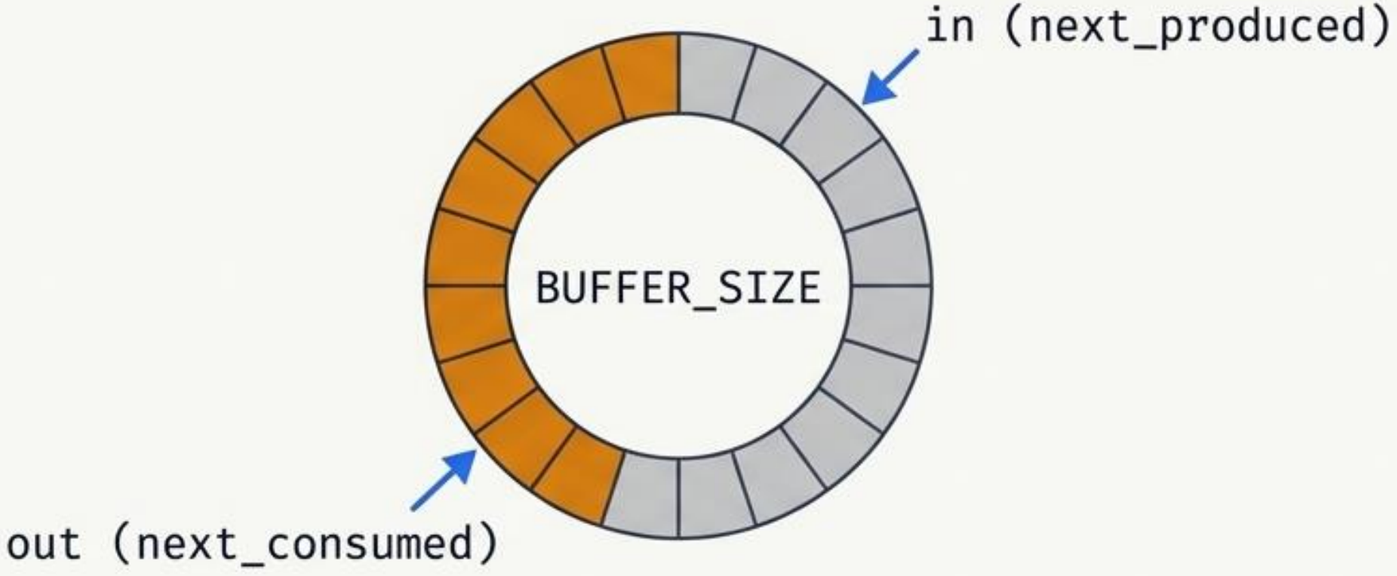
Tiêu chí	**Bộ nhớ chia sẻ** (Shared Memory)	**Trao đổi Thông điệp** (Message Passing)
Tốc độ (Speed)	Rất Cao (Truy cập trực tiếp bộ nhớ)	Thấp/Trung bình (Phải copy dữ liệu)
Can thiệp của HĐH	Chỉ lúc thiết lập vùng nhớ	Mọi thông điệp đều qua System Call
Độ phức tạp lập trình	Khó (Lập trình viên tự đồng bộ hóa)	Dễ hơn (HĐH lo việc truyền tải)
Ứng dụng lý tưởng	Dữ liệu lớn, truy cập liên tục	Dữ liệu nhỏ, phân tán, an toàn

Nhiệm vụ Thảo luận

Dựa vào bảng ma trận chuẩn hóa IPC, nhóm hãy tư vấn lựa chọn mô hình cho 2 hệ thống sau:
1) Hệ thống Chat nội bộ (gửi text ngắn gọn liên tục).
2) Trình biên tập Video (nhiều tiến trình cùng thao tác trên một mảng pixel chung). Giải thích lý do.

Bài toán kinh điển: Producer - Consumer

Vùng đệm giới hạn (Bounded-buffer). Dữ liệu được quản lý qua biến “in” và “out”.




```
while ((in + 1) % BUFFER_SIZE) == out)
    ; /* do nothing */
```

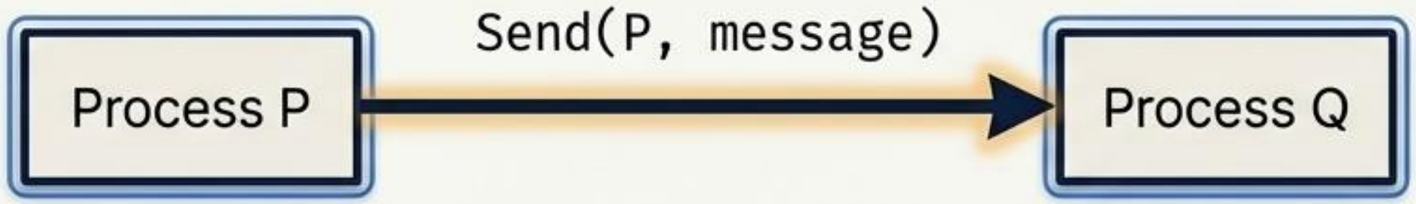
Nhiệm vụ Thảo luận

Tìm lỗ hổng logic:
Quan sát vòng lặp chờ của Producer “`((in + 1) % BUFFER_SIZE) == out)`”.

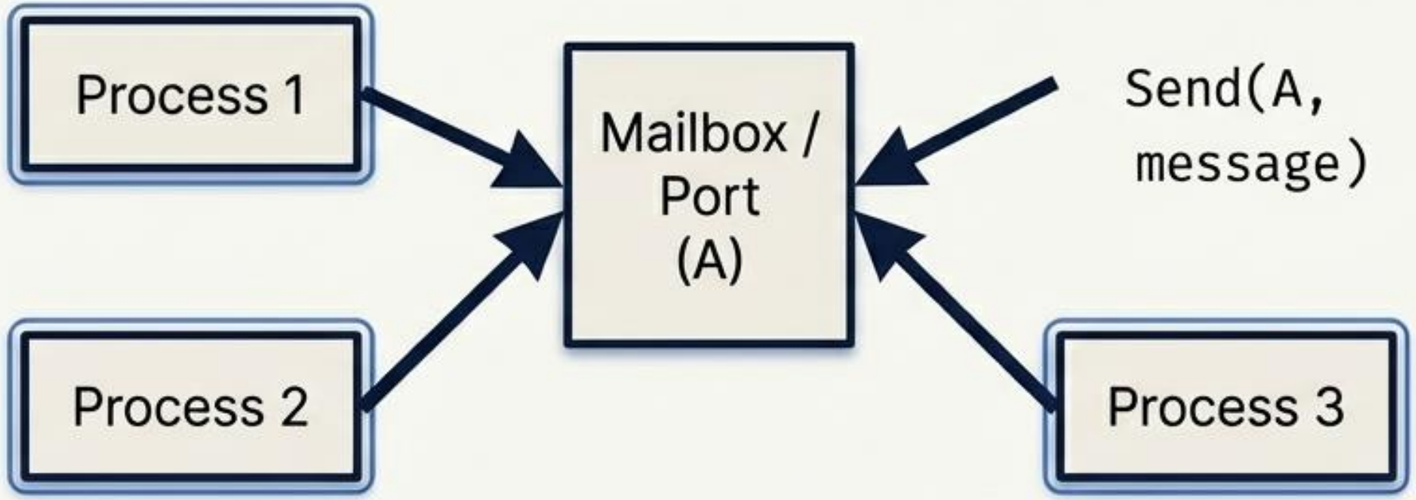
Nhóm hãy chứng minh bằng toán học hoặc vẽ sơ đồ **tại tại sao giải pháp code này chỉ cho phép lưu trữ tối đa** `BUFFER_SIZE - 1` phần tử, dù mảng có kích thước là `BUFFER_SIZE`?

Gửi Điệp tín: Trực tiếp & Gián tiếp

Truyền thông Trực tiếp



Truyền thông Gián tiếp

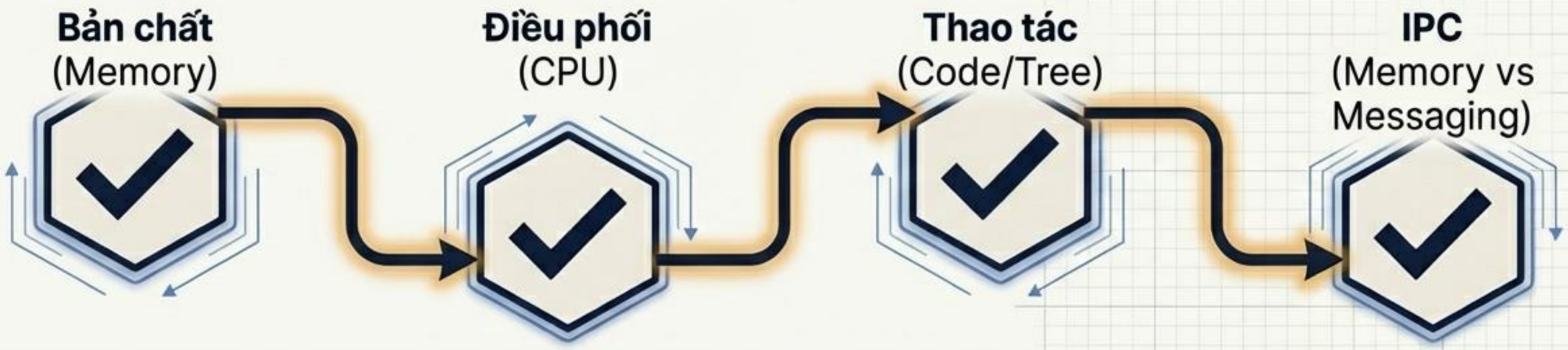


Nhiệm vụ Thảo luận

Phân tích

Phân tích ưu và nhược điểm về mặt "kiến trúc phần mềm" của việc định danh rõ ràng người nhận bằng `Send(P, message)` (Trực tiếp) so với việc ném thông điệp vào một hộp thư chung `Send(A, message)` (Gián tiếp). Cách nào linh hoạt hơn khi hệ thống mở rộng?

Tổng kết Bài tập Nhóm & Bước tiếp theo



Chỉ thị từ Giảng viên:

- Tổng hợp các lập luận giải quyết "**Nhiệm vụ Thảo luận**" thành báo cáo kỹ thuật (1 trang/nhóm).
- Nộp mã nguồn mô phỏng tiến trình phân nhánh (`fork/wait`) lên hệ thống môn học.
- Chuẩn bị cho Chương 4: Xem lại lỗ hổng của bài toán Bounded-buffer (Chuyên đề 13). Chúng ta sẽ dùng "**Đồng bộ hóa**" (Synchronization) và "Mutex/Semaphores" để vá lỗ hổng này vào tuần sau.